

Creating an App

In this tutorial we will be making a simple ParaPy application. Firstly we will set up a layout of the application containing placeholders. We build a ParaPy model representing a staircase. Then we will build a user interface that allows user to configure the properties of the staircase.

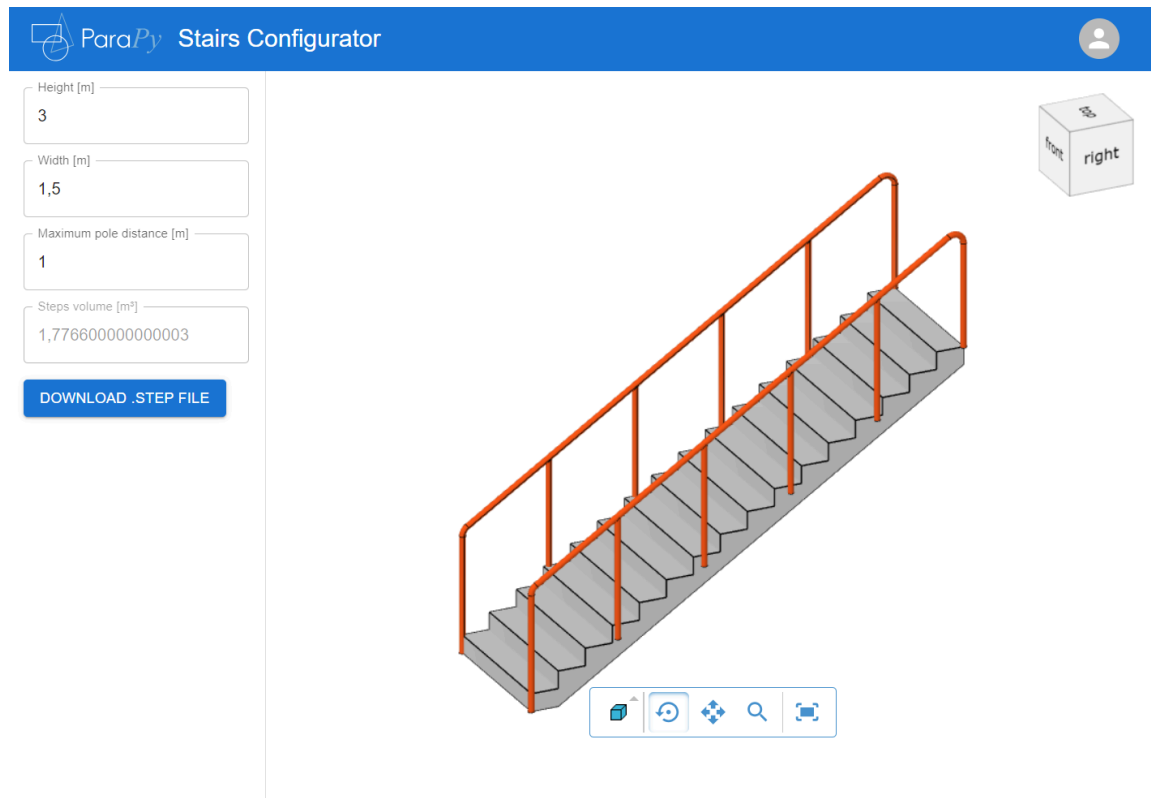


You will learn

- Setting up an project
- Creating a layout for an application
- Creating a geometric parapy model
- Integrating a model with the UI
- Generating and downloading a .step file

Idea

The idea of the app is to create a parametric staircase. The goal of the application to quickly generate a staircase STEP file based on some parameters. Below is an image of the final app we will be making in this tutorial.



Approach

To get started we will first make a rough layout of the application with placeholders which we will fill in later. We will then start making the model of the stairs. The model will contain the rules to generate the geometry of the stairs. We will then connect the model with the UI so that users of the app can edit the properties of the stairs. The changing of the properties will influence the generation of the geometry and the user will see the updated stairs. We finalize the app by adding a option to export the stairs to a .step file.

Base app

In a new python script, we will start by creating our `App` component. The root component that will render the entire application. That component can then be displayed using the `display` function. When the script is executed, a webpage should open displaying ``Hello world'.

Note that since we are developing an application, we set `reload=True` in the `display` function. When we now make changes to the script, the application will directly be updated. Try it out by changing the text returned in the `render` method.

```
from parapy.webgui.core import Component, NodeType

class App(Component):
```

```
def render(self) -> NodeType:
    return 'Hello world'

if __name__ == "__main__":
    from parapy.webgui.core import display
    display(App, reload=True)
```



Reload

In the `display` function we set `reload` to `True`. When this is set, the application will reload whenever it detects changes in the source code. This allows for a nicer development experience.

The reload function is an experimental feature and might not always work as intended. In these cases a regular application restart is required to see the changes.

For the remainder of this tutorial it is assumed that hot reload is turned on.

We are now ready to develop our application.

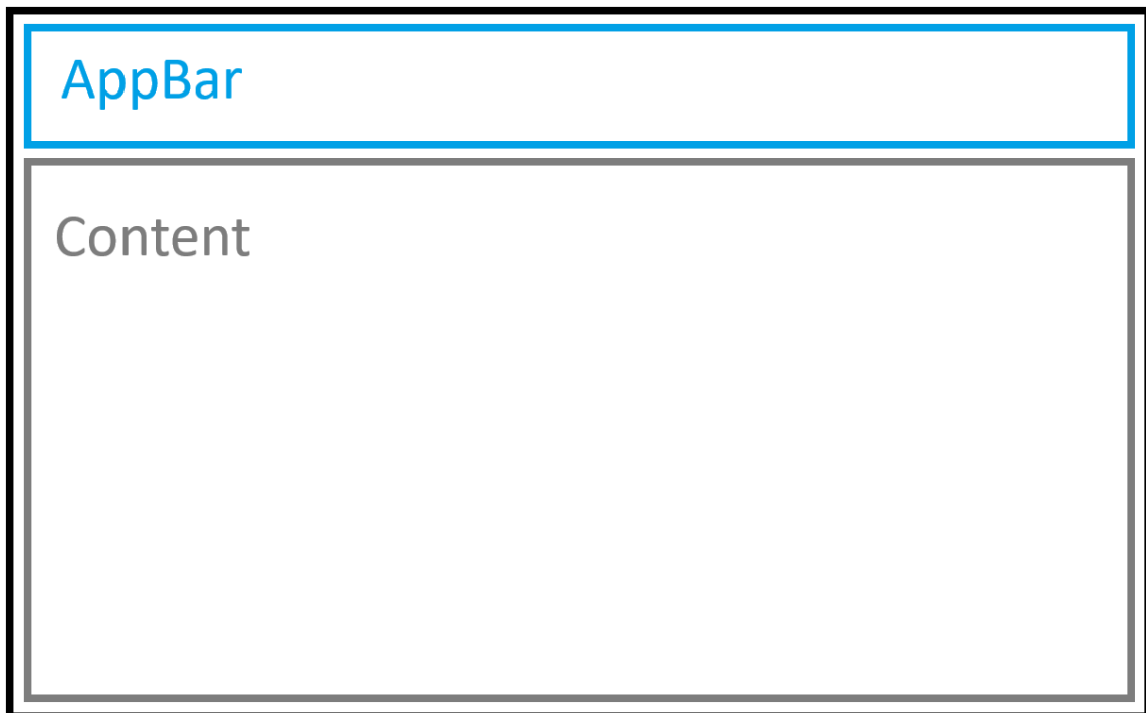
App layout

Let's start by defining the layout of our application. If we look at the app we want to make, we notice 3 main components.

- The app bar
- The inputs panel
- The model viewer

Now we have to figure out how we divide the available area of the application between these three components.

The first division we can see is that the page is vertically divided between the **app bar** and both the **inputs panel** and the **model viewer**. Schematically we can draw this division as follows:



Now we have a plan for our first step of the layout. That is to split the app into two sections above each other. One being the `AppBar` and the other being the `content`. To achieve this in code we can use the `Split` widget from the `layout` package. This widget allows you to split areas into sections.

Layout package

Make sure that the `layout` package is installed with

```
pip install parapy-webgui-layout
```



Then the package can be imported using:

```
from parapy.webgui import layout
```



Let's try it out by adding the split widget with some placeholders:

```
class App(Component):  
    def render(self) -> NodeType:  
        return layout.Split[  
            'AppBar',  
            'content'  
        ]
```



As you can see the desired effect is not quite achieved. By default, the `Split` widget splits the area horizontally. We can specify that the content should be split vertically instead:

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical')[
            'AppBar',
            'content'
        ]
```

Now it looks like our changes have the desired effect. However, this is not exactly true. Let's inspect the `Split` to make sure that it is dividing the entire web page for our two sections. We can achieve this by temporarily adding a background color:

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                           style={'backgroundColor': 'gray'})[
            'AppBar',
            'content'
        ]
```

The result is unexpected. We would like the entire webpage to be in our area. However only the two most upper rows are colored. This is caused because a `Split` by default only takes the minimal required height. Since we currently only have two rows of text, the `Split` will only be that tall. We would like for the split to occupy the entire web page. We can do exactly that by providing `height="100%"` to the `Split`. A height of `"100%"` means that the `Split` will be as tall as 100% of the parent. In our case the parent is the entire web page since the `Split` is the first widget we render.

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                           height='100%',
                           style={'backgroundColor': 'gray'})[
            'AppBar',
            'content'
        ]
```

When we apply the above changes, we can see that the split occupies the entire page like we wanted. However, the second text is now in the middle of the page. This is because `Split` divides the available area in sections. By default all sections are equally large. In our case the first section containing the `app bar` should only be as high as the `app bar` itself.

We can achieve this by providing the `weights` of each section. The `weights` controls how the available area is divided between the sections. If `weights=[1, 2]` would be provided to the `Split`, the second section would be twice as large as the first. Additionally we can provide a weight of `0` to a section. This means that the section should be as small as its content allows. This is exactly what we need for the `app bar`. Thus we want a `appbar` with weight `0` and the other should be as high as the remainder:

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1],
                             style={'backgroundColor': 'gray'})[
            'AppBar',
            'content'
        ]
```

Now that we have the desired result we can again remove the background color and add the actual **app bar** instead of our placeholder. ParaPy provides an `AppBar` which contains some nice build in features to work with the cloud. The widget can be imported using:

```
from parapy.webgui.app_bar import AppBar
```

AppBar

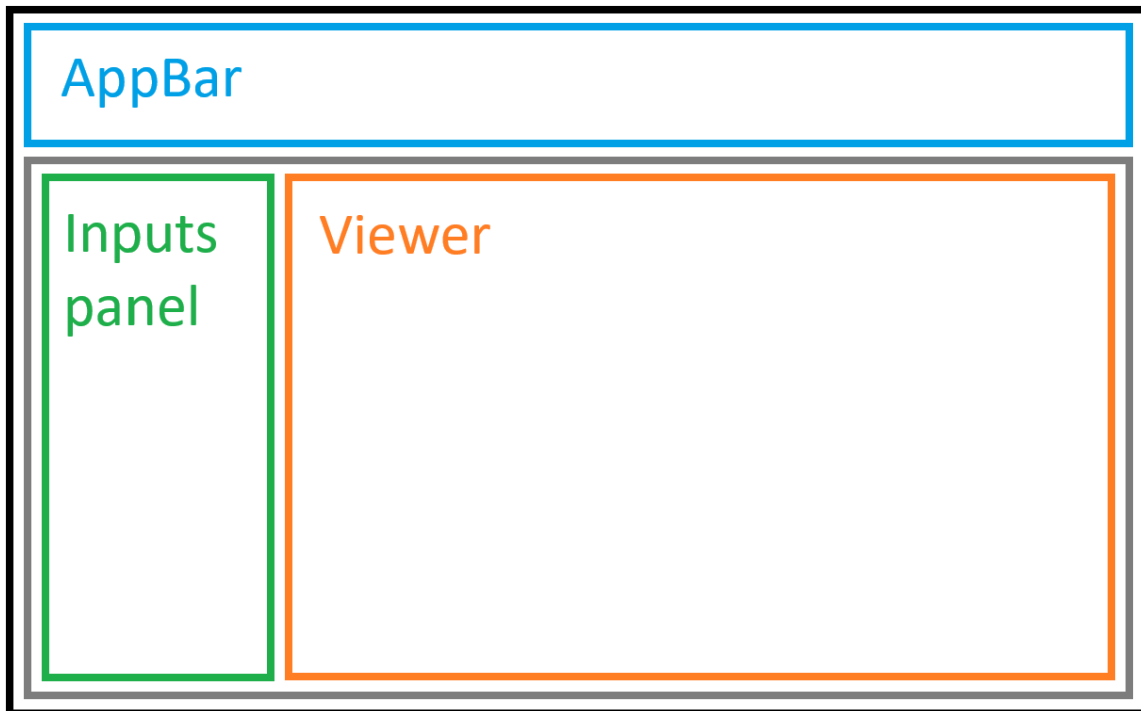
The AppBar widget is provided by default with the parapy-webgui package. There is no need to install any additional packages.

We can also provide the appbar with a title to display.

```
from parapy.webgui.app_bar import AppBar

class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1])[
            AppBar(title="Stairs Configurator"),
            'content'
        ]
```

When we now check the result, we have a nice `AppBar` and a placeholder for the content. Now that we know how a `Split` works we can use the same technique to split the `content` in two sections for the **inputs panel** and the **model viewer**.



In this case we want to split the available area horizontally. The **inputs panel** should be as small as its content whilst the **model viewer** should be as big as possible to make use of the real estate.

```
from parapy.webgui.app_bar import AppBar

class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1])[
            AppBar(title="Stairs Configurator"),
            layout.Split(height='100%',
                         weights=[0, 1])[
                'inputs panel',
                'model viewer'
            ]
        ]
```

As the results show, the inputs panel placeholder text and the model viewer placeholder text are next to each other. It would be nice to have a visual separator between the two panels. The MUI package contains a nice widget to achieve this. A **Divider** widget draws a nice visual separator. It can be controlled to be either horizontal and vertical.

To use this widget we first have to modify our split. We are now going to split it into 3 sections instead of 2. The first for the **inputs panel**, the second for the **Divider** and the third for the **model viewer**. We have to modify the **weights** as well. The number of **weights** should always match up with the number of sections. So we need 3 weights. The **Divider** also should be as small as possible. So we change the **weights** to `[0, 0, 1]`:

```
layout.Split(height='100%',
             weights=[0, 0, 1])[
    'inputs panel',
    'Divider placeholder',
    'model viewer'
]
```

We can now replace the divider placeholder with the actual widget. We can obtain the `Divider` from the `mui` package. The `mui` package is a package that contains a lot of stylized widgets.

MUI package

Make sure that the `mui` package is installed with

```
pip install parapy-webgui-mui
```

Then the package can be imported using:

```
from parapy.webgui import mui
```

We have to specify that the `Divider` should be vertically oriented, since by default it is horizontal.

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1])[
            AppBar(title="Stairs Configurator"),
            layout.Split(height='100%',
                         weights=[0, 0, 1])[
                'inputs panel',
                mui.Divider(orientation='vertical'),
                'model viewer'
            ]
        ]
```

We now have finished the initial layout of the app. We still have to replace the placeholder for the `inputs panel` and the `model viewer`.

Inputs panel

For the inputs panel, we will create a new component. Evidently we already have quite some code inside our `App` component. It is good practice to create a component for each section to make reusable, readable and performant web apps. Up until now we have just used a single

component. Arguably we could have made a separate component for `content`. However, to not overcomplicate this tutorial we will only make a separate component for the `inputs panel`.

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1])[
            AppBar(title="Stairs Configurator"),
            layout.Split(height='100%',
                         weights=[0, 0, 1])[
                InputsPanel,
                mui.Divider(orientation='vertical'),
                'model viewer'
            ]
        ]

class InputsPanel(Component):
    def render(self) -> NodeType:
        return 'Input panel placeholder'
```

We don't know exactly yet what we want to display in the inputs panel since we don't have our model yet. So we will park the development of the inputs panel until we know more about our model.

Model viewer

Let's replace the `model viewer` with a geometry viewer widget. The `viewer` package contains a widget named `Viewer`. This widget allows the displaying of geometrical shapes.

Viewer package

Make sure that the `viewer` package is installed with

```
pip install parapy-webgui-viewer
```

Then the package can be imported using:

```
from parapy.webgui import viewer
```

Let's replace the placeholder with this widget:

```
class App(Component):
    def render(self) -> NodeType:
        return layout.Split(orientation='vertical',
                             height='100%',
                             weights=[0, 1])[
            AppBar(title="Stairs Configurator"),
```

```
layout.Split(height='100%',  
            weights=[0, 0, 1])[  
    InputsPanel,  
    mui.Divider(orientation='vertical'),  
    viewer.Viewer,  
]
```

When we now check our app, we can see a viewer has appeared. Nothing is being shown because we have not yet told the viewer what it should display. The viewer receives an `objects` argument that allows the user to provide objects that should be displayed. For now we will just add a `Cube` so we can check that it works.

Import `Cube` from `parapy.geom` and provide it to the `objects` argument

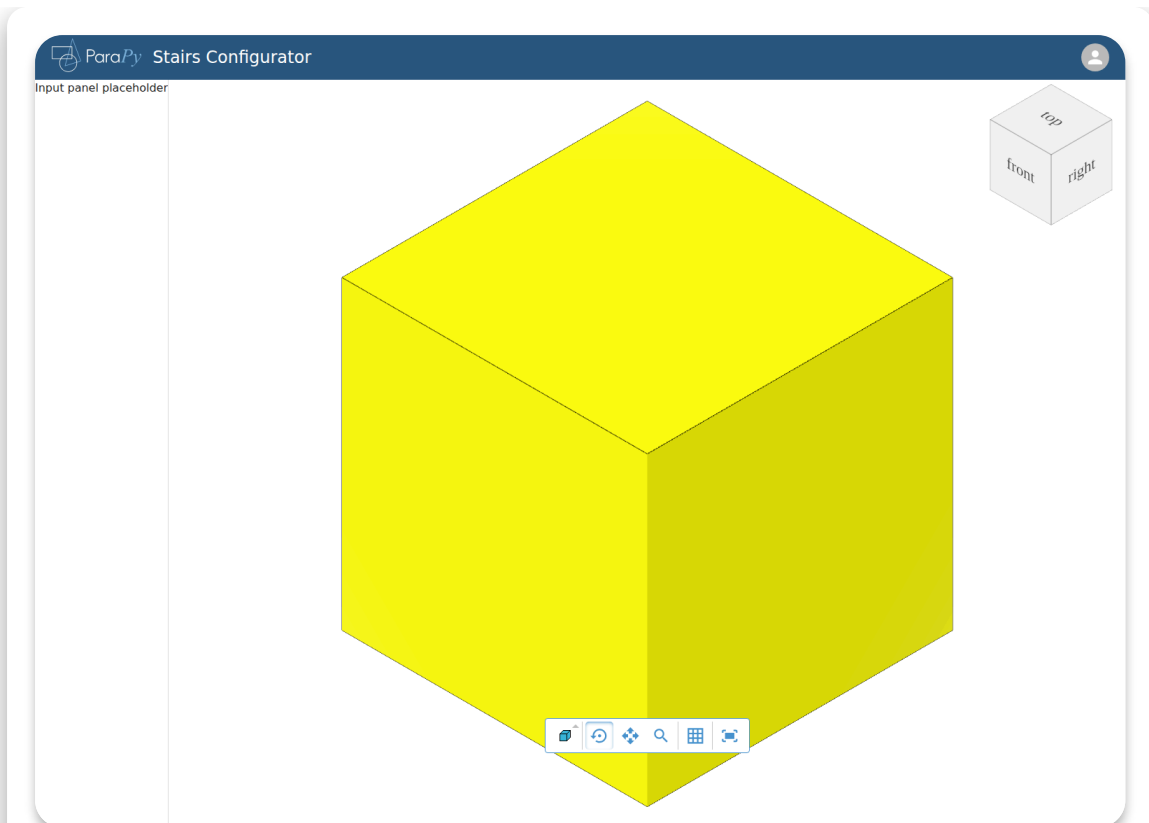
```
from parapy.geom import Cube  
...  
viewer.Viewer(objects=Cube(1))  
...
```

A cube should now be visible in the viewer. We will replace this cube shortly with our model.

Summary

You have now seen how you can divide the app in sections with `layout.Split`. You can then provide widgets from different libraries to create webpages. We will complete the `inputs panel` later once we have a model to control.

The code app so far looks like this:



This is a preview of the demo, to get a live version [install the documentation locally](#)

Expand code



```

1 class App(Component):
2     def render(self) -> NodeType:
3         return (
4             layout.Split(orientation='vertical',
5                           height='100%',
6                           weights=[0, 1])[
7                 AppBar(title="Stairs Configurator"),
8                 layout.Split(height='100%',
9                               weights=[0, 0, 1])[
10                     InputsPanel,
11                     mui.Divider(orientation='vertical'),
12                     viewer.Viewer(objects=Cube(1)),
13                 ]
14             ]
15         )
16
17
18 class InputsPanel(Component):
19     def render(self) -> NodeType:
20         return 'Input panel placeholder'

```

Creating a model

We want to make a model of a stairs. The stairs should consist of the steps and a railing on each side. To create this model we will make use of the ParaPy modeling language, as well as

use the ParaPy geometry library.



ParaPy models

This tutorial assumes that you have an entry level knowledge of the ParaPy modeling language. For more details on the modeling language, check out the [ParaPy core tutorial](#)

Let's start by defining our model:

```
from parapy.core import Base

class Stairs(Base):
    pass
```

Now that we have a model, we will be defining the rules of that module using ParaPy Slots. There are three types of Slots:

- **Input** Slots are used to define the inputs of a model.
- **Attribute** Slots are used to define properties of a model that are calculated. These calculations are done lazily and are cached allowing for significant performance boosts.
- **Part** Slots are used to define parent child relations. They allow you to define what a object consists out of.

Inputs

Since we are making a parametric design, we want to define all the parameters that are required for the stairs. We can also add more inputs as we go, but we can already think of a few inputs that are required to model a stairs:

- **height**
- **width**
- **step_height**
- **step_depth**

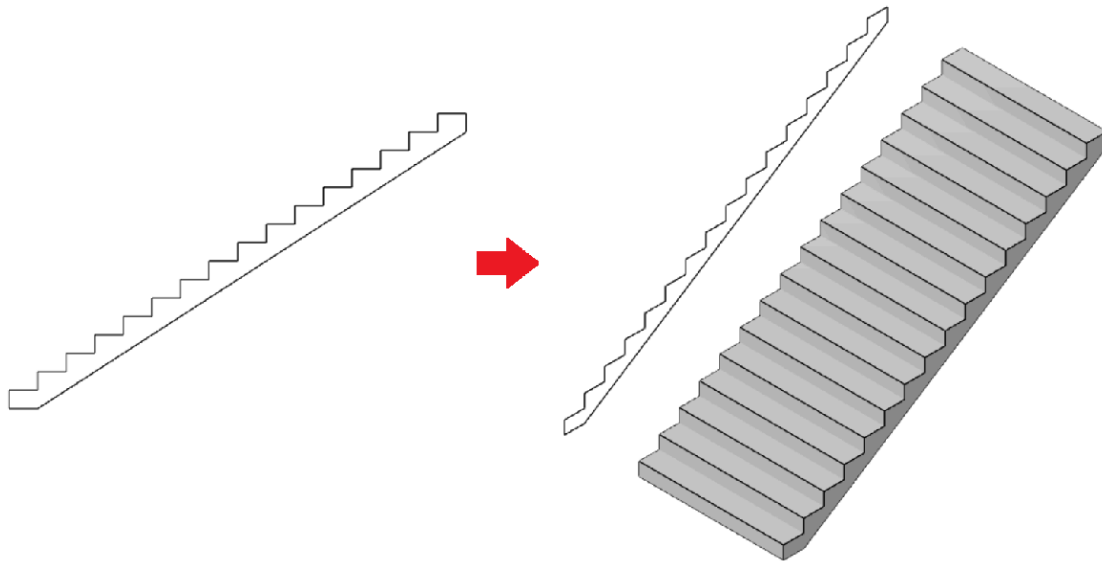
Let's add inputs for these parameters to our model. We can also guess some default values:

```
from parapy.core import Base, Input, Attribute, Part

class Stairs(Base):
    height: float = Input(5)
    width: float = Input(1.5)
    step_height: float = Input(0.180)
    step_depth: float = Input(0.280)
```

The steps

To add the geometry for the steps to our model, we first need to have a strategy to create the geometry. There are multiple ways to achieve the desired geometry. Most likely the easiest approach is an extrusion. If we look at the stairs we can clearly see that the side profile of the stairs is just extruded along the width of the stairs. All we have to do is define the side profile outline and then extrude it:

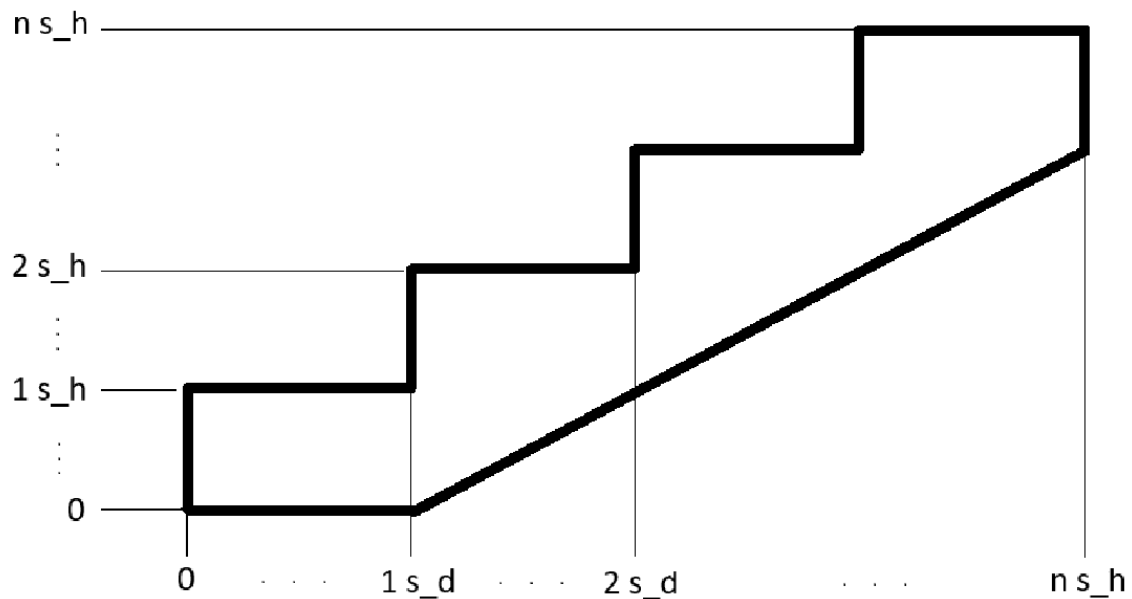


First let's start by defining the points of the polygon. We can do this by making a `Attribute` in the model that returns a list of the points:

```
class Stairs(Base):
    ...

    @Attribute
    def steps_outline_points(self):
        pts = [...]
        return pts
```

We now have to come up with an algorithm that gets all the points. We will start by defining the first point. For this we select the most bottom left point and decide that this is the origin $(0, 0, 0)$. Then for each step we will add 2 points. Both these points have a z value of the $(\text{step_index} + 1) * \text{step_height}$. The y value of the first point is $\text{step_index} * \text{step_depth}$. The second will have a y value of $(\text{step_index} + 1) * \text{step_depth}$.



Before we can make these point we first need to know how many steps are there in the stairs. So let's make a `Attribute` for that:

```
class Stairs(Base):
    ...

    @Attribute
    def number_of_steps(self):
        return int(self.height / self.step_height)
```

As you can see from the equation, we decide to round down the number of steps. Now that we have the number of step we can add points for each step in our outline `Attribute`. We now only have to add 2 more points that outline the backside of the stairs. We can see the coordinates in the scheme above for these points, namely: $(0, n * s_d, (n-1) * s_h)$ and $(0, s_d, 0)$.

Point

The `Point` class is used. This can be imported from the `parapy.geom` package:

```
from parapy.geom import Point
```

The final algorithm looks like:

```
@Attribute
def steps_outline_points(self):
    pts = [Point()]
    s_h = self.step_height
    s_d = self.step_depth
```

```

n = self.number_of_steps

# add 2 points for each step
for step_number in range(n):
    z = (step_number + 1) * s_h
    y1 = step_number * s_d
    y2 = (step_number + 1) * s_d
    pts.append(Point(y=y1, z=z))
    pts.append(Point(y=y2, z=z))

# add 2 points for the backside of the steps
pts.append(Point(y=n * s_d, z=(n - 1) * s_h))
pts.append(Point(y=s_d))
return pts

```

Let's now draw a line through them. For this we can use the `Polygon` from `parapy.geom`. We can thus add a `Part` to our model, the `steps_outline`:

```

from parapy.geom import Polygon

class Stairs(Base):
    ...

    @Part
    def steps_outline(self):
        return Polygon(points=self.steps_outline_points)

```

Now that we have the outline. It would be nice to see it in the viewer we created earlier. If we create an instance of our stairs class, we can give it to the viewer to display:

```

class Stairs(Base):
    ...

# Stairs instance
STAIRS = Stairs()

class App(Component):
    ...
    viewer.Viewer(objects=STAIRS)
    ...

```

The outline should now be visible. The points are not displayed because it is an `Attribute`. The Viewer by default only shows the `Part` slots of an object. If you would like to also display the points you can provide them to the `objects` argument of the `Viewer` manually:

```
viewer.Viewer(objects=[STAIRS, STAIRS.steps_outline_points])
```

Now that we have an outline, let's extrude it. For this we add another `Part` to our model. Since we want to make a solid we can use the `ExtrudedSolid` class from `parapy.geom`. As input, we

provide the island to extrude, the direction and distance. For the direction we provide a `Vector` which points to the `x` direction.

ExtrudedSolid and Vector

The `ExtrudedSolid` and `Vector` classes are used. These can be imported from the `parapy.geom` package. An in depth look on how the `ExtrudedSolid` class works can be found [here](#).

```
from parapy.geom import ExtrudedSolid, Vector
```

The solid can be added as such:

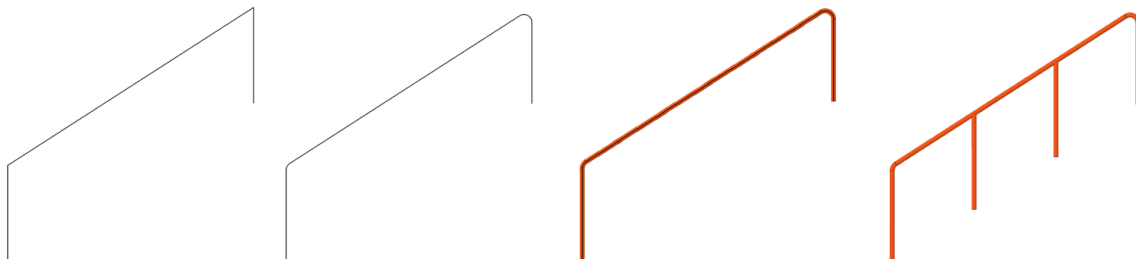
```
class Stairs(Base):
    ...

    @Part
    def steps(self):
        return ExtrudedSolid(island=self.steps_outline,
                             direction=Vector(x=1),
                             distance=self.width,
                             color=(200, 200, 200))
```

Now remember that we added the entire model to the `Viewer` earlier. The extrusion is now part of the model, so the viewer should update and the extruded steps should be visible.

The hand rails

To build up the rails, we will first start by defining a path for the handrail. We will create this path by defining a polygonal wire. We will then fillet the corners to allow for the sweeping of a profile. To finish the handrail section, we will sweep a circular profile through the defined line. The supports are cylinders distributed over the length of the stairs. Finally, we will fuse the cylinders and handrail to make 1 single piece. That piece can then be mirrored to the other side of the stairs.



Let's start with the polygon defining the path of the handrail. We will have to provide a list of points to the polygon. The start and end point we have calculated before for the steps outline.

The two upper points of the rail are the same points, but translated upwards with a set distance. We can add a new `Input` named `railing_height` for this variable.

One additional note is that a `Polygon` by default closes the path. We have to tell it to not connect the last point with the first point. We can do this by setting `force_closure=False`.

```
class Stairs(Base):
    ...
    railing_height: float = Input(1.25)
    ...
    @Part
    def rough_rail_path(self):
        return Polygon(
            points=[
                Point(),
                Point(z=self.railing_height),
                Point(y=self.number_of_steps * self.step_depth,
                    z=self.number_of_steps * self.step_height +
self.railing_height),
                Point(y=self.number_of_steps * self.step_depth,
                    z=self.number_of_steps * self.step_height),
            ],
            force_closure=False
        )
```

The next step would be to fillet this polygonal wire we just created. We can use the `FilletWire` from `parapy.geom` to achieve this. Create a new `Part` that returns a `FilletWire` which receives the polygon we created as an argument. The second argument is the radius with which we would like to round the corners of the polygon.

```
from parapy.geom import FilletWire

class Stairs(Base):
    ...
    @Part
    def rail_path(self):
        return FilletWire(built_from=self.rough_rail_path,
                        radius=0.1)
```

The result will be a nicely rounded wire.

At this point the viewer will get starting cluttered with all the parts that we are making. That is because we are telling the `Viewer` widget to display the entire model. We can give it a subset of our model instead to have fine control of what should be displayed. In this case we would like to see the newly created filleted wire, but hide the rough polygon. To achieve this the `Viewer` widget inputs would look like this:

```
viewer.Viewer(objects=[STAIRS.steps, STAIRS.rail_path])
```

The viewer can always be adjusted to inspect what we are currently making.

Now that we have our path, we can create a profile and sweep it along the path to get our handrail. The `SweptSolid` receives a path and a profile as input. For the profile we can make a `Circle`.

```
from parapy.geom import Circle, SweptSolid

class Stairs(Base):
    ...

    @Part
    def rail_profile(self):
        return Circle(radius=0.03)

    @Part
    def rail(self):
        return SweptSolid(path=self.rail_path,
                           profile=self.rail_profile)
```

The result is our desired handrail path. Now the only thing missing are the supports. To get the supports we first have to make a distribution along the stairs where they should be placed. What we can do is create a `LineSegment` between the start and end point of our handrail path. We can then get an equally spaced point distribution on that line. If we do that we have the location of each of the support piles.

To calculate the number of poles we would like to place, we introduce a new `Input` named `max_pole_distance`. This input will limit the maximum distance between the support poles. We then simply divide the length of the line, add one and round it down.

A `LineSegment` has a method `equispaced_points`. This method returns a list of points equally spaced over the line. These inputs will include the start and end point, but we can strip those since our handrail already has a vertical feature at those locations. Lastly we can make sure that at least one support pole is always present by adding `n = max(3, n)`

```
from parapy.geom import LineSegment

class Stairs(Base):
    max_pole_distance: float = Input(1.)
    ...

    @Part
    def slope_line(self):
        return LineSegment(start=self.rail_path.start,
                           end=self.rail_path.end)

    @Attribute
    def pole_points(self):
        n = int(self.slope_line.length / self.max_pole_distance + 1)
        n = max(3, n)
```

```
return self.slope_line.equispaced_points(n)[1:-1]
```

Now we can place a cylinder on each of these points with the height equal to the `railing_height`. We have to specify that the `Part` we are creating is not just a single cylinder, but a multitude. For this we will use `quantify=len(self.pole_points)`. This means that the number of cylinders created will be equal to the length of the `pole_points` list. For the position of the cylinder we can create a `Position` object which receives for its location the point. To use the index of the "current" cylinder, we can use `child.index`. `child` can be imported from `parapy.core`.

child

`child` is a unique ParaPy object which can be used in `@Part` expressions. They allow you to reference the part that is generated. It is particularly useful for `@Part` expressions that use `quantify`, where `child` is the instance of the object that is created. You can then, for example, use `child.index` to get the index of the object currently being created.

```
from parapy.geom import Cylinder

class Stairs(Base):
    ...

    @Part
    def poles(self):
        return Cylinder(quantify=len(self.pole_points),
                        height=self.railing_height,
                        radius=0.03,
                        position=Position(self.pole_points[child.index]))
```

We now have all the subsections of our rail. Let's fuse it into a single solid. `FusedSolid` from `parapy.geom` does exactly this. It receives a shape and a number of tools to fuse on that shape.

```
from parapy.geom import FusedSolid

class Stairs(Base):
    ...

    @Part
    def railing_left(self):
        return FusedSolid(shape_in=self.rail,
                          tool=self.poles,
                          color=(230, 80, 20))
```

We now have our completed rail on the left. We can now mirror this rail to the right using `MirroredShape`, also imported from `parapy.geom`. We have to give the axis and reference point along which the shape is mirrored.

```

from parapy.geom import MirroredShape

class Stairs(Base):
    ...

    @Part
    def railing_right(self):
        return MirroredShape(shape_in=self.railing_left,
                               reference_point=Point(x=self.width / 2),
                               vector1=VY,
                               vector2=VZ,
                               color=(230, 80, 20))

```



VY and VZ

`VY` and `VZ` are used. These are predefined unit vectors in the `Y` and `Z` axis. They can be imported from `parapy.geom`. Alternatively you could use the `Vector` object to create these vectors:

```

vector1=Vector(0, 1, 0),
vector2=Vector(0, 0, 1),

```

And there we have it. The final stairs are complete. Let's make sure the viewer is displaying only the three final objects:

```

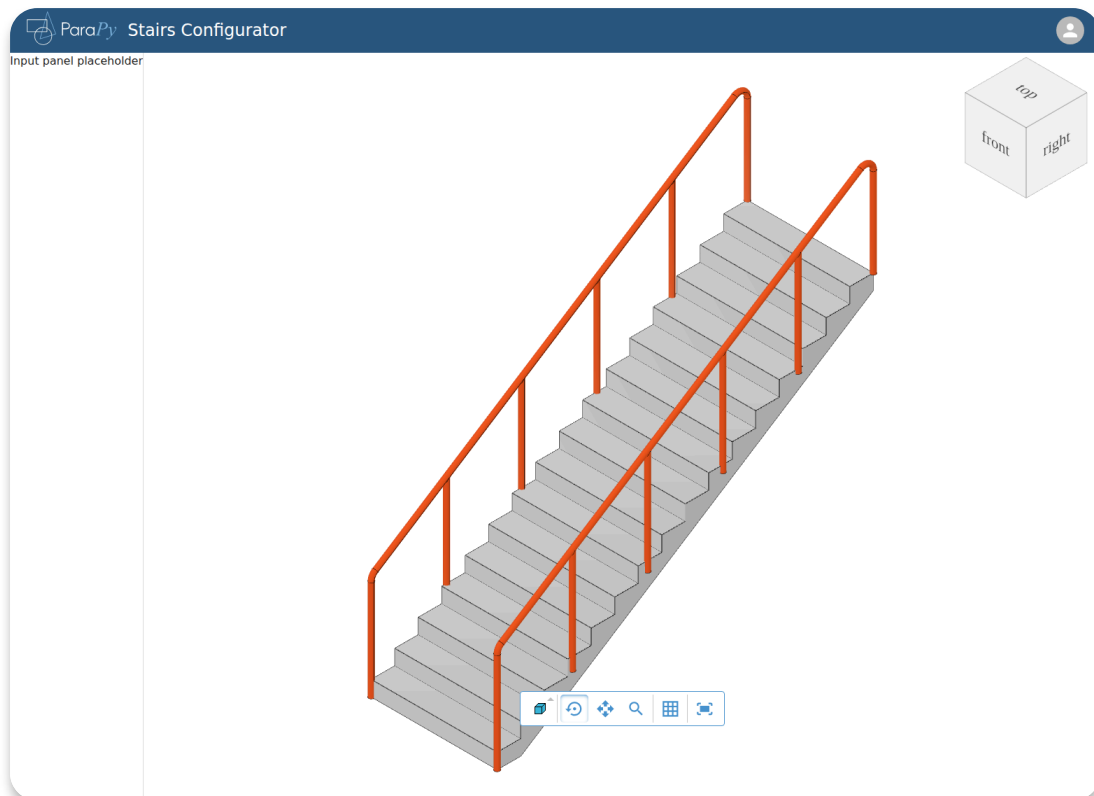
viewer.Viewer(objects=[STAIRS.steps, STAIRS.railing_left, STAIRS.railing_right])

```

Summary

We used `parapy.core` to create a model. We filled this model with inputs, attributes and parts describing the model. To describe the geometry of the stairs we used some tools from the `parapy.geom` library. The model has been displayed in the viewer, by creating a global instance and providing the parts that we would like to show to the `Viewer` widget.

The code of the app so far looks like this:



This is a preview of the demo, to get a live version [install the documentation locally](#)

Expand code



```

1  class Stairs(Base):
2      height = Input(3)
3      width = Input(1.5)
4
5      step_depth = Input(0.280)
6      step_height = Input(0.180)
7      max_pole_distance = Input(1)
8
9      railing_height = Input(1.25)
10
11     @Attribute
12     def number_of_steps(self):
13         return int(self.height / self.step_height)
14
15     @Attribute
16     def steps_outline_points(self):
17         pts = [Point()]
18         s_h = self.step_height
19         s_d = self.step_depth
20         n = self.number_of_steps
21         for step_number in range(n):
22             z = (step_number + 1) * s_h
23             y1 = step_number * s_d
24             y2 = (step_number + 1) * s_d
25             pts.append(Point(y=y1, z=z))
26             pts.append(Point(y=y2, z=z))
27         pts.append(Point(y=n * s_d, z=(n - 1) * s_h))
28         pts.append(Point(y=s_d))

```

```
29         return pts
30
31     @Part
32     def steps_outline(self):
```

Input panel

Now that we have a model, we can complete our app by filling in the `inputs panel` placeholder.

Let's start by creating a plan for the panel. What we want is a number of fields below each other, each controlling an input. Additionally, we want a button that -when clicked- downloads a STEP file of the stairs.

Input panel layout

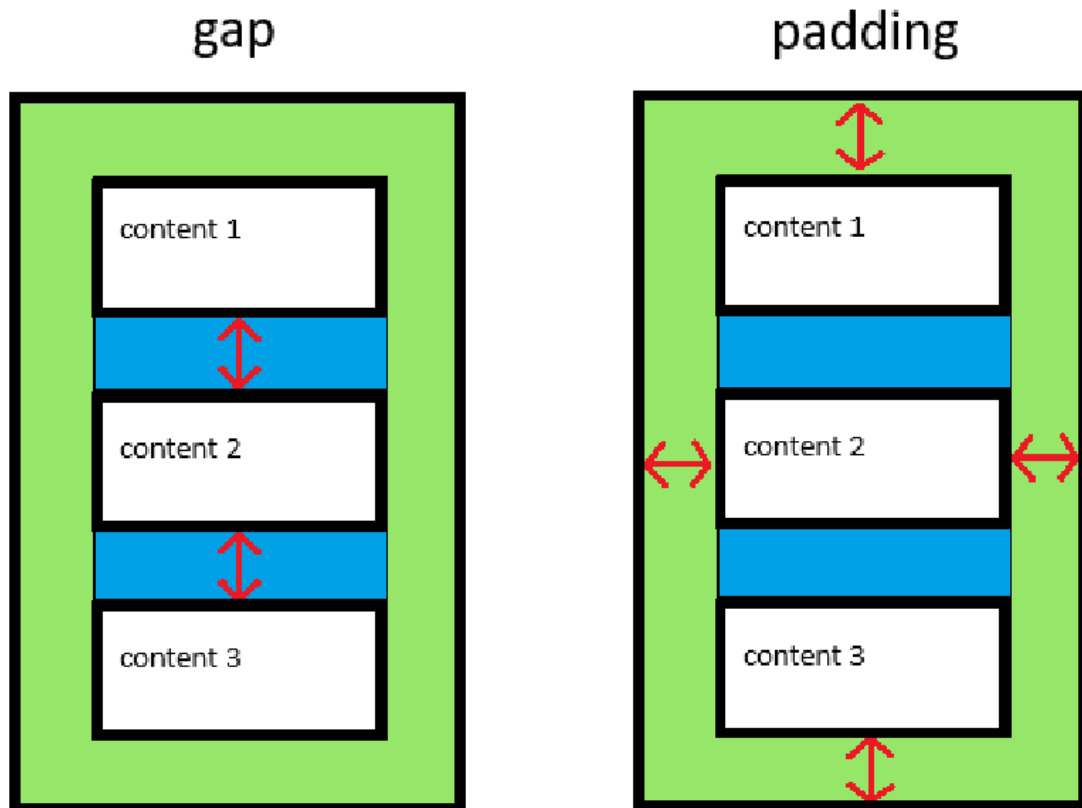
The layout of the input panel is straightforward. It represents a vertical list of widgets.

Previously we used `Split` to divide the area into sections. In this case we do not want to split the input panel area, but list the content. For this `Box` from the `parapy.webgui.layout` package can be used. The `Box` widget is similar to the `Split` widget. The difference is that the `Split` has weights to allow for the dividing of the available space, while `Box` simply lists its children.

Let us replace the render function of our `InputsPanel` component with a `Box`. For the content of the box, we can use the `TextField` widget from the `parapy.webgui.mui` package. This widget is simply an input field.

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical')[
            mui.TextField,
            mui.TextField,
            mui.TextField,
        ]
```

When we look at the results, the text fields are very close to each other. There is also no spacing between the text fields and the border. We can adjust the styling of our `Box` to add these spacings. Specifically we want to change the `gap` value and the `padding` of the `Box`.



For spacing between the children we can set the `gap` attribute. This accepts [any css value](#) in string format. For our case, let's set the gap between the text fields to `1em`, which is equal one text line height.

We now have spacing, but we would also like some padding to the border. For this we will have to give the `Box` css styling. To give css styling to a widget, we can use the `style` argument. This accepts a dictionary with css properties. In this case we want to add padding, so we will use `style={"padding": "1em"}`.

The final Box styling looks like this:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            mui.TextField,
            mui.TextField,
            mui.TextField,
        ]
```

We now have the layout of the input panel as we would like. The next step is to make it so that the text fields update the model.

Controlling the model

In the previous step we added some text fields. Now we just have to update the model whenever the user changes these fields.

There are some different approaches to achieve this.

One approach is to provide the textfield with an event handler. An event handler is a function that is called whenever the event fires. In this case, we could listen to an `onChange` event. So whenever the user changes the value, the function is called. In this function we can use the new value to update the model. This is possible, but in this case we would have to do some manual processing of the value, since they will be returned as strings. There are also some edge cases that might be difficult to handle.

A second approach is to use pre-fabricated widgets for interacting with parapy models. The `layout` package exposes a series of widgets for this exact purpose. These widgets are called `SlotField` widgets. There are a variety of them for different type of values.

Let's use the second approach for our problem since we are dealing with a ParaPy model here. We can replace the first text field with a `SlotFloatField`. We can then link it to one of the inputs. For example the `height` input. A `SlotField` widget expects a model and a slot name to which it should be linked:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            layout.SlotFloatField(STAIRS, 'height'),
            mui.TextField,
            mui.TextField,
        ]
```

The first input will now be a slot field. Try changing the value and then clicking away. The model displayed in the viewer should update everytime the user is done editing the value (clicks away). Now we can replace the remaining two textfields with other slots we would like to control. For example `width` and `max_pole_distance`:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            layout.SlotFloatField(STAIRS, 'height'),
            layout.SlotFloatField(STAIRS, 'width'),
            layout.SlotFloatField(STAIRS, 'max_pole_distance'),
        ]
```

Now we have control over these three inputs. We can also reference some other parts from our model that are not inputs, but outputs. The `SlotField` will then become non-editable and only

displays the value. Let's add a field displaying the volume of the stairs. We can get the volume directly from the steps extrusion using `STAIRS.steps.volume`. We can then directly pass this to a display field like `mui.TextField`:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            layout.SlotFloatField(STAIRS, 'height'),
            layout.SlotFloatField(STAIRS, 'width'),
            layout.SlotFloatField(STAIRS, 'max_pole_distance'),
            mui.TextField(value=STAIRS.steps.volume)
        ]
```

When we look at the labels of the slots, we see that by default the name of the attribute is used. It would be nice to give custom labels by providing the `label` argument:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            layout.SlotFloatField(STAIRS, 'height', label='Height [m]'),
            layout.SlotFloatField(STAIRS, 'width', label='Width [m]'),
            layout.SlotFloatField(STAIRS, 'max_pole_distance', label='Maximum pole
distance [m]'),
            mui.TextField(value=STAIRS.steps.volume, label='Steps volume [m³]')
        ]
```

Whilst the result is what we were looking for, it would be nice to have the labels in front of the fields. So let's take a look on how to do this. The `mui` package has a lot of styled widgets, including widgets for the organization of forms. We can use a `FormControl` to make a group for each field. Then we can add a `FormLabel` widget to that group:

```
class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box(orientation='vertical',
                           gap='1em',
                           style={'padding': '1em'})[
            mui.FormControl[
                mui.FormLabel['Height [m]'],
                layout.SlotFloatField(STAIRS, 'height'),
            ],
            mui.FormControl[
                mui.FormLabel['Width [m]'],
                layout.SlotFloatField(STAIRS, 'width'),
            ],
            mui.FormControl[
                mui.FormLabel['Maximum pole distance [m]'],
                layout.SlotFloatField(STAIRS, 'max_pole_distance'),
            ],
        ]
```

```

        mui.FormControl[
            mui.FormLabel['Steps volume [m³]'],
            mui.TextField(value=round(STAIRS.steps.volume, 3)),
        ],
    ]

```

If you check the result, it is almost what we were hoping for. There are now two overlapping labels. Let's tell the fields that they should not show a label by setting `show_label` to `False`:

```

...
layout.SlotFloatField(STAIRS, 'height', show_label=False),
...

```

STEP file download

The goal is to generate a step file from the stairs and allow the user to download it. We can get started by adding a button. For a button it is recommended to use the `Button` widget from the `parapy.webgui.mui` package. The `Button` widget has a variety of arguments to allow for customization. For a full overview of what you could do with a `Button` check out the [mui documentation](#).

In our case we would like a `Button` of the `contained` variant. We can also give it some text as its child:

```

class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box[
            ...
            mui.Button(variant='contained')[
                "Download .STEP file"
            ]
        ]

```

Now that we have a button let's attach a function to it which we would like to be called whenever the user presses the button. To achieve this we can provide a function with an event which we would like to listen to. For the button we would like to listen to the `onClick` event:

```

class InputsPanel(Component):
    def render(self) -> NodeType:
        return layout.Box[
            ...
            mui.Button(variant='contained',
                       onClick=self.download_step)[
                "Download .STEP file"
            ]
        ]

    def download_step(self, evt):
        print('button clicked!')

```

Let's take a closer look at this piece of code. First give an `onClick` argument to the button. We provide `onClick` with the function `self.download_step`. Note that we give the function, we are **not** calling the function:

- wrong: `onClick=self.download_step()`
- correct: `onClick=self.download_step`

The button, whenever pressed, will look if there is a `onClick` argument. Then the function provided will be called. Thus whenever the user presses the button right now, the `download_step` method will be called. This method right now only print something to the console.

A second thing to note it that the function receives an argument `evt`. This argument contains information about the event. Right now we don't plan to do anything with it. For textfield events this `evt` value contains the value of the textfield for example.

Our goal now is to replace this printing with the behaviour of generating a step file and then let the user download it.

Let's first generate a step file. To do this we can use the `STEPWriter` class from the `parapy.exchange` package. This class allows for the writing of a `.step` file given some geometry. In our case we want only the `step`, `railing_left` and `railing_right` to be written to the step file. So lets provide that as a list to the writer:

```
class InputsPanel(Component):  
    def download_step(self, evt):  
        writer = STEPWriter([STAIRS.steps,  
                             STAIRS.railing_left,  
                             STAIRS.railing_right])
```

We now have a `STEPWriter` ready to write a `.step` file. We just have to provide it with a `file_path` to write to.

Since we want the file to be downloaded by the user, we have to write it to the assets directory. The assets directory is a directory whose content can be accessed by users through the web. All other files cannot be accessed by the user for security reasons. So firstly let's check if we have an assets directory.

By default, the WebGUI will look for an assets directory next to the file that is being executed:

```
.  
├── assets/  
│   └── stairs.step  
└── app.py
```

If this folder doesn't exist, then the WebGUI will not find it. You will have to create this folder for the WebGUI to automatically recognize it.

Another option is to tell the WebGUI specifically the location of the assets directory. In the `display` function we can specify this value:

```
if __name__ == '__main__':
    from parapy.webgui.core import display

    display(App, reload=True, assets_dir='/path/to/assets/dir')
```

It is however recommended to use the first option of auto discovery.

Now that we have created an assets directory we can write our step file to that directory:

```
from parapy.webgui.core import get_assets_dir

...

class InputsPanel(Component):
    ...

    def download_step(self, evt):
        writer = STEPWriter([STAIRS.steps,
                             STAIRS.railing_left,
                             STAIRS.railing_right])

        assets_dir = get_assets_dir()
        filename = os.path.join(assets_dir, 'stairs.step')

        writer.write(filename)
```

Now whenever the button is clicked the `stairs.step` file should appear in the assets directory. The only thing left is to tell to the webpage that the file should be downloaded.

We can use the `download_file(url)` function provided by the `parapy.webgui.core.actions` package to trigger this download. We just have to obtain a `url` to the file in the assets directory. To get this we can use the `get_asset_url` function. This function returns a url that points towards a file in the assets directory. All together this is what the function looks like:

```
from parapy.webgui.core import get_assets_dir, get_asset_url
from parapy.webgui.core.actions import download_file

...

class InputsPanel(Component):
    def render(self) -> NodeType:
        ...

    def download_step(self, evt):
        writer = STEPWriter([STAIRS.steps,
                             STAIRS.railing_left,
                             STAIRS.railing_right])
```

```

assets_dir = get_assets_dir()
filename = os.path.join(assets_dir, 'stairs.step')

# write file to assets dir
writer.write(filename)

# get url and instruct to download the file behind the url
url = get_asset_url(filename)
download_file(url)

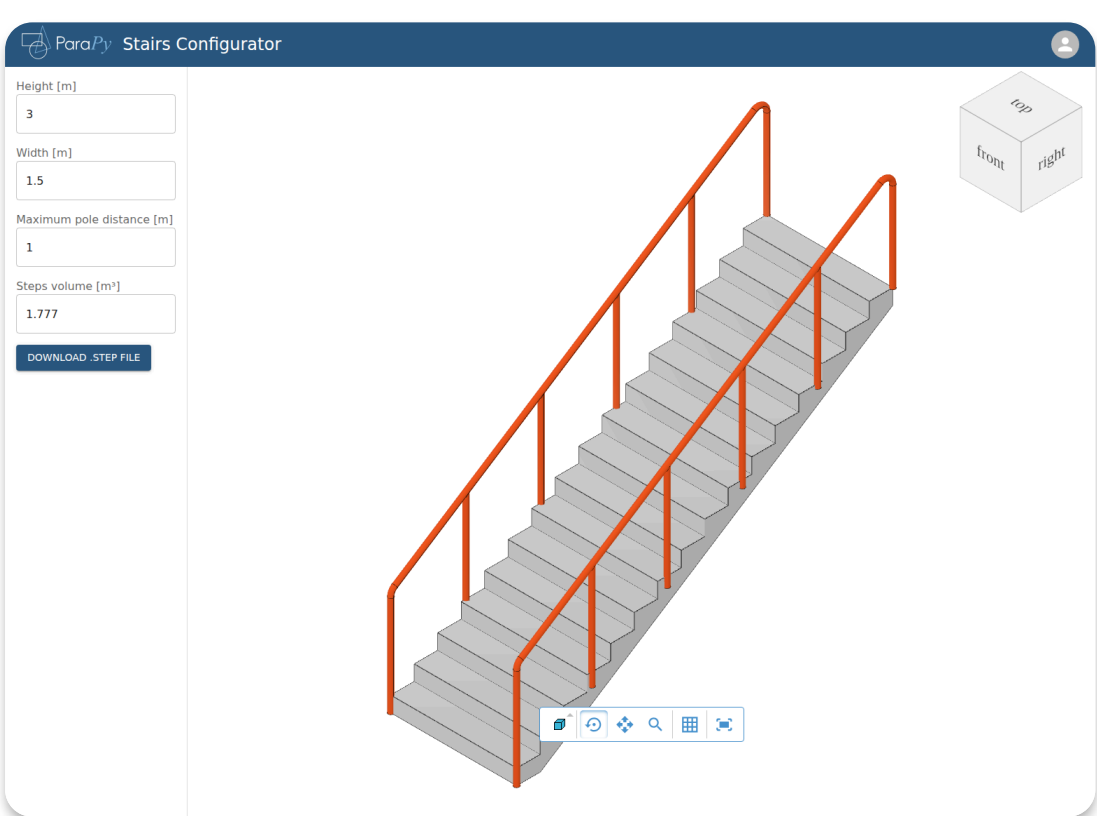
```

We now have a button that when clicked, downloads a step file of the current stairs model.

Summary

The app now has an inputs panel whereby the user can control the stairs model. We use `SlotFields` to directly interface with ParaPy models.

The code app so far looks like this:



This is a preview of the demo, to get a live version [install the documentation locally](#)

[Expand code](#)

```

1 class Stairs(Base):
2     height = Input(3)
3     width = Input(1.5)
4
5     step_depth = Input(0.280)

```

```

6      step_height = Input(0.180)
7      max_pole_distance = Input(1)
8
9      railing_height = Input(1.25)
10
11     @Attribute
12     def number_of_steps(self):
13         return int(self.height / self.step_height)
14
15     @Attribute
16     def steps_outline_points(self):
17         pts = [Point()]
18         s_h = self.step_height
19         s_d = self.step_depth
20         n = self.number_of_steps
21         for step_number in range(n):
22             z = (step_number + 1) * s_h
23             y1 = step_number * s_d
24             y2 = (step_number + 1) * s_d
25             pts.append(Point(y=y1, z=z))
26             pts.append(Point(y=y2, z=z))
27         pts.append(Point(y=n * s_d, z=(n - 1) * s_h))
28         pts.append(Point(y=s_d))
29         return pts
30
31     @Part
32     def steps_outline(self):

```

Reflection

There are some aspects in the code we could improve on.

Take a look at how the UI gets access to the model. With the current implementation we define a global instance of the model. Then each component directly grabs this instance whenever it is needed. This approach becomes problematic when making reusable components in a growing app. Right now none of the components we have created are truly reusable, since we would always have to change the pointer to the stairs. If we ever were to create another app that needs this inputs panel, we would also have to define a global stairs variable. This is not according to the WebGUI philosophy.

This can be fixed by giving all components a `Prop`. Props are inputs for classes. When we give this prop we can provide components with values. In our case we would give a instance of `Stairs` to the `App` component:

```

class App(Component):
    stairs: Stairs = Prop()

    ...

if __name__ == '__main__':
    from parapy.webgui.core import display

```

```
STAIRS = Stairs()
display(App(stairs=STAIRS), reload=True)
```

Now in the render function, we can pass this `self.stairs` to the child components that also need it:

```
class App(Component):
    stairs: Stairs = Prop()

    def render(self) -> NodeType:
        return (
            layout.Split(orientation='vertical',
                          height='100%',
                          weights=[0, 1])[
                AppBar(title="Stairs Configurator"),
                layout.Split(height='100%',
                              weights=[0, 0, 1])[
                    InputsPanel(stairs=self.stairs),
                    mui.Divider(orientation='vertical'),
                    viewer.Viewer(objects=[self.stairs.steps,
                                           self.stairs.railing_left,
                                           self.stairs.railing_right]),
                ]
            ]
        )

class InputsPanel(Component):
    stairs: Stairs = Prop()
    ...

if __name__ == '__main__':
    from parapy.webgui.core import display

    STAIRS = Stairs()
    display(App(stairs=STAIRS), reload=True)
```

Now the app and inputs panel components are reusable when it comes to the `Stairs` instance.

What is next?

Next we would like to deploy our app to the cloud so that users can use it. When everything is set up, it is just a single command to push your app to the cloud. To learn more about the cloud go to the [cloud tutorial](#).